

Benchmarking Continuous and Discrete-Time PINNs on the Three-Dimensional Beltrami Flow

Duarte Miguel Fradinho Fernandes Malveiro (87181)

duarte.f.malveiro@tecnico.ulisboa.pt

PhD in Electrical and Computer Engineering

Institute of Systems and Computer Engineering - Research and Development, Lisbon, Portugal

September 2, 2026

Abstract—In the present work, various implementations of Physics-Informed Neural Networks (PINNs) are used to solve the three-dimensional (3D) Beltrami flow, which is a particular case of the Navier-Stokes equations. These implementations are the continuous-time or vanilla PINNs (V-PINNs) and discrete-time PINNs (DT-PINNs), each of which with and without Adaptive Loss Weights (ALW) — fixed weights and variable weights, respectively — besides varying the initial learning rate (η_0) and activation function. The network inputs are the spatial and temporal coordinates (for V-PINNs) or the spatial coordinates (for DT-PINNs) of the respective domains, and the outputs are the predictions for the velocity components and pressure. The goal is to minimize the loss function, whose components are the residuals of the Navier-Stokes equations, and the squared difference between the predictions and the analytical solutions of the 3D Beltrami flow, imposed as boundary and initial conditions for the problem. The models are trained in a single GPU. The convergence tests show that with V-PINNs, the loss functions generally converge to lower values when the initial learning rate is smaller (up to an average total loss of 6.34×10^{-5}), that combining variable weights with sigmoid activation function leads to poor convergence, and that the implementation of ALW results in higher loss values, for most cases. With DT-PINNs, the training shows a chaotic behaviour for most fixed weights + sigmoid tests, and for some fixed weight + tanh tests, where local minima for average losses can surpass 10^5 . The best convergence results (for each sequential PINN) occur for the variable weights tests. The accuracy results reflect the convergence ones, where, for V-PINNs, the most accurate cases are trained with fixed weights + sigmoid + $\eta_0 = 10^{-3}$, reaching the lowest ten-run average L_2 relative error for velocity v of 3.54×10^{-3} . For DT-PINNs, the most accurate models were trained with variable weights, with the lowest ten-run average L_2 relative error for velocity w of 2.50×10^{-3} . In general, the most accurate DT-PINN models were more accurate than the most accurate V-PINN ones, but the latter consumed less computational resources than the former, especially memory.

Index Terms—Physics-Informed Neural Networks, Navier-Stokes, Training, Accuracy.

I. INTRODUCTION

The Navier-Stokes equations are partial differential equations (PDEs) that govern fluid flow motion and are key to many engineering applications. These equations include a non-linear term, which makes it impossible to establish general analytical solutions for them. Thus, to solve these equations, numerical techniques like finite differences, finite volume, and spectral methods have been used extensively in the field of study known as Computational Fluid Dynamics (CFD). However,

the underlying physical phenomena and the domain geometries or other characteristics can be highly complex, implying the generation of complex meshes and a large number of internal nodes when using these classical models, significantly raising the computational cost. Besides, the intricate nature of CFD, together with the high complexity that some problems achieve, may require huge codes, making them more prone to errors and harder to maintain.

In an attempt overcome these issues, there have been developed, in the latest years, machine learning techniques to solve many PDE equations, including Navier-Stokes. One of these techniques is Physics-Informed Neural Networks, introduced by Karniadakis [1], [2] in 2017. These differ from traditional neural networks by the inclusion of physical constraints in the loss function, namely, the governing equations, boundary conditions (BCs) and initial conditions (ICs). The enforcement of physical characteristics is key so that the model's predictions are physically realistic, unlike pure neural network models which rely exclusively on not physically-influenced data.

PINNs' inference times (after training) are substantially lower than running simulations with conventional numerical methods [3], [4]. However, unlike the classical numerical models, PINNs require training, which usually is a more computationally heavy task due to the intricate nature of the physics-constrained loss terms, which greatly reduces their competitiveness against traditional solvers.

While PINNs, in their standard form, known as continuous PINNs or vanilla PINNs (V-PINNs), may already achieve reasonable accuracy on predicting some physics phenomena, there is still significant room for improvement in that regard, especially in applications requiring high accuracy, and that involve highly complex, non-linear and transient physics, such as fluid mechanics, governed by the Navier-Stokes equations.

This work aims to explore, benchmark and compare two approaches targeting improved accuracy, and measure their effect on memory usage and training times, compared to V-PINNs, on the 3D Beltrami flow. One approach, which implies important changes on the physics modeling, is known as Discrete-Time PINNs (DT-PINNs), which consists on discretizing the time domain, using sequentially placed PINNs, one per time step, which predict the solutions in the 3D space domain at each time instant, thus respecting temporal causality, where V-PINNs tend to underperform when predicting the solutions

of time-dependent equations [5]. The other approach is not structural like the first one and is known as Adaptive Loss Weights and can be implemented in the standard V-PINNs and DT-PINNs models. Instead of the weights associated with the loss function terms being constant throughout the whole training process, with this approach, these weights will be dynamically updated based on gradient statistics during training, to balance each loss term contribution to the total loss; the imbalances between loss terms that tend to develop in the training processes may hurt model accuracy.

The Beltrami flow, which is a particular case of the Navier-Stokes equations, is, in fact, a mathematical idealization, where the flow behaviour is linear. This flow is an adequate choice for the present work, since it has analytical solutions, unlike the general Navier-Stokes equations, thus, eliminating the difficulty of finding reference solutions, which could unnecessarily hinder the learning process involved at this initial stage of my PhD, while still simulating the complexity of solving the Navier-Stokes equations.

This work is organized as follows. After Introduction, in the second section, it is presented the problem formulation and important theoretical concepts, including the Navier-Stokes equations and the Beltrami flow analytical solutions. Also, a general overview of the PINNs architecture and main operations is presented, followed by continuous and discrete-time PINNs. The third section consists on problem implementation, where the ALW technique is explained, the hardware and software frameworks, and the problem setup and metrics used to evaluate accuracy, convergence and performance of the implemented models. In the fourth section, lies the results and discussion, where accuracy and convergence for V-PINNs and DT-PINNs with and without ALW will be evaluated and explained, comparing the results between different approaches and hyperparameter combinations, containing a subsection dedicated to comparing, specifically, the configuration that leads to the best accuracy in V-PINNs and in DT-PINNs. Besides, the performance of the models tested will be evaluated, by measuring the peak memory usage and the average wall-clock time per iteration. In the fifth and final section, there are presented the main conclusions on the work performed and future work remarks.

II. PROBLEM FORMULATION

A. Navier-Stokes Equations

The Navier-Stokes equations are based on three fundamental physical principals: conservation of mass, represented by the continuity equation (Eq. (1)), Newton's second law applied to a continuum, and Newton's law of viscosity, both of which combine to form the momentum equations (Eq. (2)) [6]. The velocity-pressure form of the incompressible Navier-Stokes equations is [7]:

$$\nabla \cdot \mathbf{u} = 0, \quad (1)$$

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \frac{1}{\text{Re}} \nabla^2 \mathbf{u}, \quad (2)$$

where t is the nondimensional time, $\mathbf{u}(\mathbf{x}, t) = (u, v, w)$ is the nondimensional velocity, $\mathbf{x} = (x, y, z)$ are the three-dimensional space coordinates, and p is the non-dimensional

pressure. The relation between inertial forces and viscous forces acting at the flow is given by the Reynolds number, $\text{Re} = UL/\nu$, being L and U , respectively, the characteristic length and velocity of the flow, and ν is the kinematic viscosity of the fluid [6], [7].

Solving the Navier-Stokes equations means finding parameters (u, v, w, p) , given (x, y, z, t) , at a certain space and time domains, that satisfy Eqs. (1), (2). There is no general analytical solution for these equations, which are often solved using numerical methods, in CFD.

Beltrami flows are a particular case that satisfies the condition in Eq. (3) [8]:

$$\mathbf{u} \times (\nabla \times \mathbf{u}) = 0. \quad (3)$$

The non-linear term $(\mathbf{u} \cdot \nabla) \mathbf{u}$ of Eq. (2) can be rewritten as (Eq. (4)):

$$(\mathbf{u} \cdot \nabla) \mathbf{u} = \nabla \left(\frac{\|\mathbf{u}\|^2}{2} \right) - \mathbf{u} \times (\nabla \times \mathbf{u}), \quad (4)$$

where $\|\mathbf{u}\| = \sqrt{u^2 + v^2 + w^2}$ [6]. Given the Beltrami flow condition (Eq. (3)), the nonlinearity of the momentum equation vanishes, making the Navier-Stokes equations much simpler to solve. Ethier and Steinman [8] developed the analytical solutions for the three-dimensional unsteady Beltrami flow, where $C = -c^2$ (Eq. (5-8)):

$$u(\mathbf{x}, t) = -a [e^{ax} \sin(ay + cz) + e^{az} \cos(ax + cy)] e^{Ct} \quad (5)$$

$$v(\mathbf{x}, t) = -a [e^{ay} \sin(az + cx) + e^{ax} \cos(ay + cz)] e^{Ct} \quad (6)$$

$$w(\mathbf{x}, t) = -a [e^{az} \sin(ax + cy) + e^{ay} \cos(az + cx)] e^{Ct} \quad (7)$$

$$p(\mathbf{x}, t) = -\frac{a^2}{2} \left[e^{2ax} + e^{2ay} + e^{2az} + 2 \sin(ax + cy) \cos(az + cx) e^{a(y+z)} + 2 \sin(ay + cz) \cos(ax + cy) e^{a(z+x)} + 2 \sin(az + cx) \cos(ay + cz) e^{a(x+y)} \right] e^{2Ct} \quad (8)$$

The physical problem to solve is thus defined by Eqs. (1) and (2), where the BCs and ICs are, respectively, defined by Eqs. (5-8) at the spatial domain boundary, $\partial\Omega_s$, and at $t = 0$.

B. Physics-Informed Neural Networks

Figure 1 represents a scheme of a typical PINN model, in its standard format, as developed by Karniadakis [1], [2] in his original works, for solving a PDE, whose solution is $\mathbf{y} = (y_1, y_2, \dots, y_d)$ for inputs $\boldsymbol{\xi} = (\xi_1, \xi_2, \dots, \xi_p)$, defined on a domain $\Omega \in \mathbb{R}^p$, by the generic Eq. 9, constrained by a set of BCs, Eq. 10 [9]:

$$f(\boldsymbol{\xi}, \mathbf{y}, \nabla \mathbf{y}, \nabla^2 \mathbf{y}, \dots, \nabla^n \mathbf{y}) = 0, \quad \mathbf{x} \in \Omega, \quad (9)$$

$$\mathcal{B}(\boldsymbol{\xi}, \mathbf{y}) = 0, \quad \boldsymbol{\xi} \in \partial\Omega. \quad (10)$$

If the PDE is time-dependent, ICs are usually included, which can be considered as a particular case of BCs. $\mathcal{B}(\boldsymbol{\xi}, \mathbf{y})$ can be Dirichlet, Neumann, Robin, or periodic BCs. The PDE function can be dependent on inputs, outputs, or respective derivatives (denoted by ∇), up to the order n .

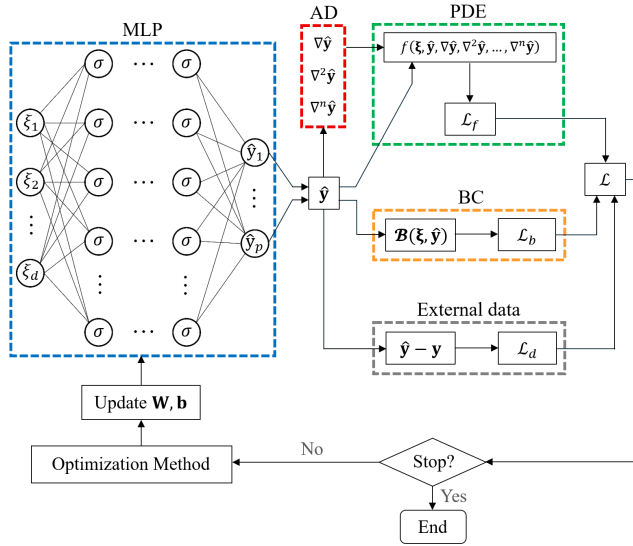


Fig. 1: Scheme of a generic PINN, illustrating the sequence performed at each iteration k . Adapted from [10].

Regarding the Fig. 1 itself, inside the blue-contoured area, lies a conventional Neural Network; more specifically, a multilayer perceptron (MLP), defined as $N^L(\boldsymbol{\xi}): \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_L}$ ($n_1 = d, n_L = p$), with L layers. Each layer $l = \{1, L\}$ contains n_l neurons, with an activation function σ , which usually is an hyperbolic tangent or a sigmoid. The weight matrix in the l -th layer is defined as $\mathbf{W}^l \in \mathbb{R}^{n_l \times n_{l-1}}$ and the bias vector is $\mathbf{b}^l \in \mathbb{R}^{n_l}$. One can define the MLP structure as below, where Eq. 11 refers to the input layer, Eq. 12 to the hidden layers, and Eq. 13 to the output layer [9]:

$$N^1(\boldsymbol{\xi}) = \boldsymbol{\xi} \in \mathbb{R}^{n_1}, \quad (11)$$

$$N^l(\boldsymbol{\xi}) = \sigma(\mathbf{W}^l N^{l-1}(\boldsymbol{\xi}) + \mathbf{b}^l) \in \mathbb{R}^{n_l}, \quad 2 \leq l < L, \quad (12)$$

$$N^L(\boldsymbol{\xi}) = \mathbf{W}^L N^{L-1}(\boldsymbol{\xi}) + \mathbf{b}^L \in \mathbb{R}^{n_L}. \quad (13)$$

which accepts $(\xi_1, \xi_2, \dots, \xi_d)$ at the input layer, and returns the predictions for the PDE solution $\hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_p)$, at the output layer. The process of predicting $\hat{\mathbf{y}}$ based on Eq. (11-13) is known as forward propagation, or inference. Any predicted variable (by the MLP) is represented with a hat, $\hat{\cdot}$.

The derivatives are computed using automatic differentiation (AD), represented in the scheme (Fig. 1) inside the red-contoured area. This AD step, where the MLP outputs are backpropagated through the network, is essential since it takes the MLP predictions and computes their derivatives in relation to the inputs, through the chain rule, to be introduced in the PDE residual, inside the green-contoured box represented as PDE. The PDE residual is the main physics-informed portion of the network, setting it apart from a conventional neural network, by enforcing the governing equations into a component $\mathcal{L}_f$ of the total loss function, $\mathcal{L}$.

Inside the orange-contoured box lies the component of the loss function related to the boundary conditions $\mathcal{L}_b$. Here, the MLP predictions are introduced into the BCs, verified in the domain boundary $\partial\Omega$.

Finally, to enrich the PINN model, one can compare the

network predictions in certain points of the domain against external data (as shown by the gray-contoured area), which may be generated by laboratory experiments, alternative models, literature, among others. This is the basis of the loss component related to the external data, $\mathcal{L}_d$.

The complete definition of the loss function can be based on the mean squared error concept (Eqs. (14-17)) [7]:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_f(\boldsymbol{\theta}) + \mathcal{L}_b(\boldsymbol{\theta}) + \mathcal{L}_d(\boldsymbol{\theta}), \quad (14)$$

$$\mathcal{L}_f(\boldsymbol{\theta}) = \frac{1}{N_f} \sum_{\boldsymbol{\xi} \in \mathcal{D}_f} \lambda_f |f(\boldsymbol{\xi}, \hat{\mathbf{y}}, \nabla \hat{\mathbf{y}}, \nabla^2 \hat{\mathbf{y}}, \dots, \nabla^n \hat{\mathbf{y}})|^2, \quad (15)$$

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{N_b} \sum_{\boldsymbol{\xi} \in \mathcal{D}_b} \lambda_b |\mathcal{B}(\boldsymbol{\xi}, \hat{\mathbf{y}})|^2, \quad (16)$$

$$\mathcal{L}_d(\boldsymbol{\theta}) = \frac{1}{N_d} \sum_{\boldsymbol{\xi} \in \mathcal{D}_d} \lambda_d |\hat{\mathbf{y}} - \mathbf{y}|^2, \quad (17)$$

where λ_f , λ_b , and λ_d are the weights (which may or may not be constants) associated with the respective loss components. The complete set of collocation points is given by $\mathcal{D} = \mathcal{D}_f \cup \mathcal{D}_b \cup \mathcal{D}_d$, where $\mathcal{D}$, $\mathcal{D}_f$, $\mathcal{D}_b$ and $\mathcal{D}_d$ denote, respectively, the total, residual, boundary and data sets, and the total number of collocation points is $N_{\text{col}} = N_f + N_b + N_d$. It is important to note that the collocation points can be training, validation or test points, depending on the stage of the model computation.

The goal of the PINN is to find parameters $\boldsymbol{\theta} = \{\mathbf{W}, \mathbf{b}\}$ that minimize $\mathcal{L}$, in a process known as "training", where a training set of points is used as inputs. After computing $\mathcal{L}$ for a given iteration, if the stopping criterion is verified, the loop stops. Otherwise, it is computed the gradient of $\mathcal{L}$ with relation to $\boldsymbol{\theta}$, $\nabla \mathcal{L}(\boldsymbol{\theta})$ (backpropagation), required to minimize the loss function $\mathcal{L}$ for gradient descent-based optimization methods. Then, an optimization method, usually based on $\nabla \mathcal{L}(\boldsymbol{\theta})$, is applied to update the MLP parameters, starting a new iteration. Similarly to the computation of the derivatives of the network outputs, the derivative operations associated with $\nabla \mathcal{L}(\boldsymbol{\theta})$ are commonly computed through AD.

There are many optimization methods available. PINNs commonly use Adam optimization (from *adaptive moment estimation*), introduced by Kingma and Ba in 2014 [11]. Adam is computationally efficient, has little memory requirements, and is well suited for problems that are large in terms of data and/or parameters, thus, being adequate for PINNs. The complete equation set for Adam is (Eq. (18-23)):

$$\mathbf{g}_k = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k), \quad (18)$$

$$\mathbf{m}_{k+1} = \beta_1 \mathbf{m}_k + (1 - \beta_1) \mathbf{g}_k, \quad (19)$$

$$\mathbf{v}_{k+1} = \beta_2 \mathbf{v}_k + (1 - \beta_2) \mathbf{g}_k^2, \quad (20)$$

$$\hat{\mathbf{m}}_{k+1} = \frac{\mathbf{m}_{k+1}}{1 - \beta_1^{k+1}}, \quad (21)$$

$$\hat{\mathbf{v}}_{k+1} = \frac{\mathbf{v}_{k+1}}{1 - \beta_2^{k+1}}, \quad (22)$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \frac{\hat{\mathbf{m}}_{k+1}}{\sqrt{\hat{\mathbf{v}}_{k+1} + \epsilon}}, \quad (23)$$

where $\mathbf{m}_{k+1}$ and $\mathbf{v}_{k+1}$ are the first and second moment estimates, respectively, β_1 and β_2 are the respective decay

rates, η_k is the learning rate at iteration k and ϵ is a very small positive constant [11].

During training, the model may also be evaluated in various sets of validation points, different from the training set, requiring a forward pass. This is based on the comparison between the validation loss and the training loss, being useful to see if the model is overfitting the training points.

Once the network is fully trained, one can predict the system solution $\hat{\mathbf{y}}$ based on a test set, also different from training and validation, which, similarly to the validation stage, only requires an inference step through the MLP.

The approach described in this subsection, which is the one used in the tests presented at this report, refers to Forward Problems, which aim to find unknown solutions of fully-known governing equations. However, there also exist Inverse Problems, whose goal is to infer unknown parameters from partially known governing equation structures or even discover the governing equations themselves from data [2], [12], but these are out of the scope of this work.

C. Continuous Physics-Informed Neural Networks

In the original studies of PINNs [1], [2], the solutions of the governing equations f are predicted via single PINN in the whole space-time domain. This implementation follows a fully-continuous approach, which is also known as vanilla PINNs (V-PINNs). When solving the Beltrami flow in a 3D space-time domain using this implementation, the PDE residual f illustrated inside the green-contoured box in Fig. 1 is, in fact, a set of four residuals, each of which being the residual of the i -th Navier-Stokes equation, $f = (f_1, f_2, f_3, f_4)$. The residuals f_i , $i = \{1, 2, 3, 4\}$ (Eqs. (24-27)), are the same as Eqs. (1) and (2), but with all terms exposed, moved to one side [7]:

$$f_1 = \frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y} + \frac{\partial \hat{w}}{\partial z}, \quad (24)$$

$$f_2 = \frac{\partial \hat{u}}{\partial t} + \hat{u} \frac{\partial \hat{u}}{\partial x} + \hat{v} \frac{\partial \hat{u}}{\partial y} + \hat{w} \frac{\partial \hat{u}}{\partial z} + \frac{\partial \hat{p}}{\partial x} - \frac{1}{\text{Re}} \left(\frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} + \frac{\partial^2 \hat{u}}{\partial z^2} \right), \quad (25)$$

$$f_3 = \frac{\partial \hat{v}}{\partial t} + \hat{u} \frac{\partial \hat{v}}{\partial x} + \hat{v} \frac{\partial \hat{v}}{\partial y} + \hat{w} \frac{\partial \hat{v}}{\partial z} + \frac{\partial \hat{p}}{\partial y} - \frac{1}{\text{Re}} \left(\frac{\partial^2 \hat{v}}{\partial x^2} + \frac{\partial^2 \hat{v}}{\partial y^2} + \frac{\partial^2 \hat{v}}{\partial z^2} \right), \quad (26)$$

$$f_4 = \frac{\partial \hat{w}}{\partial t} + \hat{u} \frac{\partial \hat{w}}{\partial x} + \hat{v} \frac{\partial \hat{w}}{\partial y} + \hat{w} \frac{\partial \hat{w}}{\partial z} + \frac{\partial \hat{p}}{\partial z} - \frac{1}{\text{Re}} \left(\frac{\partial^2 \hat{w}}{\partial x^2} + \frac{\partial^2 \hat{w}}{\partial y^2} + \frac{\partial^2 \hat{w}}{\partial z^2} \right). \quad (27)$$

The inputs of the MLP are $\boldsymbol{\xi} = (\mathbf{x}, t) = (x, y, z, t) \in \mathcal{D}$ and the outputs are $\hat{\mathbf{y}} = (\hat{\mathbf{u}}, \hat{p}) = (\hat{u}, \hat{v}, \hat{w}, \hat{p})$ at the space-time points $\boldsymbol{\xi}$. As the space-time dimensions are embedded into the PINN, the BCs and ICs indicated in the last paragraph of subsection II are included into the loss function, as, respectively, $\mathcal{L}_{BC}$ and $\mathcal{L}_{IC}$. These evaluate the velocity field predictions against the known Beltrami flow analytical

solutions (only for three of the four components — u, v, w — hence $i = \{1, 2, 3\}$), by computing the difference between each other, $\hat{u}_i - u_i$, at the j -th point located at the domain spatial boundaries, $\boldsymbol{\xi}_{BC}^{(j)}$, and at the j -th point located at $(\mathbf{x}, t = 0)$ points, $\boldsymbol{\xi}_{IC}^{(j)}$. In V-PINN, p and $\hat{p}$ are not included in $\mathcal{L}_{BC}$ and $\mathcal{L}_{IC}$, since it only appears as a gradient (∇p) in the momentum equations, being an hidden-state and thus implicitly inferred by satisfying the PDE residuals. Enforcing values for p as an IC or BC would over-restrain the problem, unnecessarily increasing the mathematical complexity (and also the computational cost) of the model without improving its accuracy.

For the V-PINN approach applied to the Beltrami flow, the loss and its components are defined as (Eqs. (28-32)) [7]:

$$\mathcal{L} = \mathcal{L}_f + \mathcal{L}_b, \quad (28)$$

$$\mathcal{L}_b = \mathcal{L}_{BC} + \mathcal{L}_{IC}, \quad (29)$$

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^4 \sum_{j=1}^{N_f} \lambda_i^f \left(f_i(\boldsymbol{\xi}_f^{(j)}) \right)^2, \quad (30)$$

$$\mathcal{L}_{BC} = \frac{1}{N_{BC}} \sum_{i=1}^3 \sum_{j=1}^{N_{BC}} \lambda_i^{BC} \left(u_i(\boldsymbol{\xi}_{BC}^{(j)}) - \hat{u}_i(\boldsymbol{\xi}_{BC}^{(j)}) \right)^2, \quad (31)$$

$$\mathcal{L}_{IC} = \frac{1}{N_{IC}} \sum_{i=1}^3 \sum_{j=1}^{N_{IC}} \lambda_i^{IC} \left(u_i(\boldsymbol{\xi}_{IC}^{(j)}) - \hat{u}_i^j(\boldsymbol{\xi}_{IC}^{(j)}) \right)^2, \quad (32)$$

where λ_i are the weights associated with each loss term, N_f , N_{BC} and N_{IC} are, respectively, the number of training points for residual, BC and IC losses, and $f_i(\boldsymbol{\xi}_f^{(j)})$ is the residual of the i -th equation at the j -th point. There is no $\mathcal{L}_d$ term because for the present work the predictions were not compared with external data.

D. Discrete-Time Physics-Informed Neural Networks

Space-time dynamical systems are inherently causal. This means that these systems' behaviour along time depend on the initial and boundary data, being crucial for evaluating how information propagates in the time axis. Given the non-linearity and chaotic behaviour of many physical systems defined by PDEs, such as the Navier-Stokes equations, it is hard to predict how such systems evolve. Numerical methods are employed to solve these equations, which, besides being discretized in the space domain, are also discretized in the time domain, ensuring that the solution at time t is fully resolved before passing to the next time step $t + \Delta t$, reflecting the causal structure inherent to these numerical methods [5].

V-PINN models may violate temporal causality when trying to predict time-dependent systems since these models incorporate the time dimension as a network input at the same level of any other input. There is no mechanism that forces the network to follow a temporal structure while training; moreover, according to [5], when trained by gradient descent-based methods, V-PINNs tend to first approximate PDE solutions at later times, even when initial conditions are imposed, which can lead to poor model accuracy.

The PINNs approach explained in this subchapter is known as Discrete-Time PINNs (DT-PINNs), first introduced by Chen

et al., in 2023 [13], where the time domain is discretized using a numerical scheme, with N time steps of length $\Delta t = (t_f - t_0)/(N - 1)$, considering a time domain $\Omega_t = [t_0, t_f]$.

The initial condition is applied by training a standard MLP to fit the Beltrami flow solutions at $t = 0$. The loss function is defined as in Eq. (32), except that in the first summation, $i = \{1, 2, 3, 4\}$, since the analytical solution for $p(\mathbf{x}, 0)$ and its prediction $\hat{p}(\mathbf{x}, 0)$ are also included into the loss computation. As there is no physical constraints enforcement (through the residuals), the pressure term must be included here.

For $t > 0$, the approach is as follows. The Navier-Stokes equations are a set of PDEs of the form (Eq. (33)):

$$\frac{\partial \hat{u}_i}{\partial t} - F(\hat{u}_i(\mathbf{x}, t)) = 0, \quad (33)$$

where F is a differential operator applied to $\hat{u}_i$. Equation (33) can be approximated by a second-order midpoint rule, Eq. (34) [14]:

$$\frac{\hat{u}_i^{(n+1)} - \hat{u}_i^{(n)}}{\Delta t} - F\left[\frac{\hat{u}_i^{(n+1)} + \hat{u}_i^{(n)}}{2}\right] = 0, \quad (34)$$

which is used to estimate a solution for the time step being currently computed, $\hat{u}_i^{(n+1)} = \hat{u}_i(\mathbf{x}, t + \Delta t)$, given the predicted solution for the previous time step, $\hat{u}_i^{(n)} = \hat{u}_i(\mathbf{x}, t)$. Instead of solving Eq. (33) in the whole space-time domain using a single PINN, like in V-PINNs, here the approach is to find $\hat{u}^{(n+1)}$, using a PINN that minimizes the current loss function, Eq. (35-38),

$$\mathcal{L}^{(n+1)} = \mathcal{L}_f^{(n+1)} + \mathcal{L}_b^{(n+1)}, \quad (35)$$

$$\mathcal{L}_b^{(n+1)} = \mathcal{L}_{BC}^{(n+1)}, \quad (36)$$

$$\mathcal{L}_f^{(n+1)} = \frac{1}{N_f} \sum_{i=1}^4 \sum_{j=1}^{N_f} \lambda_i^f \left(\frac{\hat{u}_i^{(n+1)}(\boldsymbol{\xi}_f^{(j)}) - \hat{u}_i^{(n)}(\boldsymbol{\xi}_f^{(j)})}{\Delta t} - F\left[\frac{\hat{u}_i^{(n+1)}(\boldsymbol{\xi}_f^{(j)}) + \hat{u}_i^{(n)}(\boldsymbol{\xi}_f^{(j)})}{2}\right] \right)^2, \quad (37)$$

$$\mathcal{L}_{BC}^{(n+1)} = \frac{1}{N_{BC}} \sum_{i=1}^4 \sum_{j=1}^{N_{BC}} \lambda_i^{BC} (u_i^{(n+1)}(\boldsymbol{\xi}_{BC}^{(j)}) - \hat{u}_i^{(n+1)}(\boldsymbol{\xi}_{BC}^{(j)}))^2, \quad (38)$$

each time step at a time, sequentially: $\hat{u}_i^{(1)} \rightarrow \hat{u}_i^{(2)} \rightarrow \dots \rightarrow \hat{u}_i^{(N-1)} \rightarrow \hat{u}_i^{(N)}$. Thus, to solve the governing equations, there are employed N PINNs, in a sequence, each having inputs $\boldsymbol{\xi} = (x, y, z) \in \mathcal{D}$ and outputs $\hat{\mathbf{y}} = (\hat{u}, \hat{v}, \hat{w}, \hat{p}) \equiv \hat{\mathbf{u}}_i = (\hat{u}_1, \hat{u}_2, \hat{u}_3, \hat{u}_4)$ at the space points $\boldsymbol{\xi}$.

Analogously to $t = 0$, in $\mathcal{L}_{BC}$, the summation from $j = 1$ to N_{BC} is performed 4 times ($i = 1, 2, 3, 4$), see Eq. (38). The reason for not including p and $\hat{p}$ in $\mathcal{L}_b$ for V-PINNs seems to be not valid for DT-PINNs, because preliminary tests showed that model accuracy improves with the inclusion of the pressure term in $\mathcal{L}_{BC}$, for $t > 0$, so it was decided to include it.

The standard MLP and all PINNs share the same network architecture, with the same number of layers and nodes per layer. This is essential to ensure continuity in the physical process throughout the time domain, accelerating the training of each PINN and improving convergence and model accuracy.

The model parameters $\boldsymbol{\theta}$ of the last epoch of the model trained at t (starting from $t = 0$) are then used as model parameters for the first epoch of the model of the next time step, $t + \Delta t$.

III. PROBLEM IMPLEMENTATION

A. Adaptive Loss Weights

The loss function of a PINN is often hard to minimize due to gradient imbalances between the different loss gradient terms that tend to develop during backward propagation. For instance, if $\nabla L_f(\boldsymbol{\theta})$ is too large compared to the gradients of the remaining loss terms, the model might find wrong solutions that accurately satisfy the governing equations, due to poor fitting to boundary and initial conditions, which can lead to inaccurate predictions [15]. Therefore, it is important to balance the contributions of the different loss gradient terms, attributing them the adequate weights, which is a cumbersome task to perform manually, since the optimal weights vary not only between different loss gradient terms, but also as the training progresses. Wang proposed in 2020 [15] a technique known as Adaptive Loss Weights, where the weights associated with the loss terms change during training depending on backpropagated gradient statistics, to compensate for the above-mentioned imbalances, aiming to stabilize training and improve model accuracy.

The total loss function can be written as (Eq. (39)):

$$\mathcal{L} = \mathcal{L}_1(\boldsymbol{\theta}) + \sum_{i=2}^{N_{\mathcal{L}}} \lambda_i \tilde{\mathcal{L}}_i(\boldsymbol{\theta}), \quad (39)$$

where, for the present physical problem, $\mathcal{L}_1$ is the loss component associated with the residual f_1 , and $\tilde{\mathcal{L}}_i$ is the unweighted loss component i (considering a total of $N_{\mathcal{L}}$ loss components). Rewriting $\mathbf{g}_k$ from Eq. (18), one has Eq. (40):

$$\mathbf{g}_k = \nabla_{\boldsymbol{\theta}} \mathcal{L}_1(\boldsymbol{\theta}_k) + \sum_{i=2}^{N_{\mathcal{L}}} \lambda_i \nabla_{\boldsymbol{\theta}} \tilde{\mathcal{L}}_i(\boldsymbol{\theta}_k). \quad (40)$$

The strategy implemented at this report to adaptively update the weights λ_i during training is based on a modified version of the formula proposed by Wang [15], similar to the alternative developed by Karniadakis in 2020 [7]. At each training iteration, $k + 1$, a first estimation for the weights, $\tilde{\lambda}_{i,k+1}$, is computed as (Eq. (41)):

$$\tilde{\lambda}_{i,k+1} = \frac{|\nabla_{\boldsymbol{\theta}} \mathcal{L}_{1,k+1}|}{|\nabla_{\boldsymbol{\theta}} \mathcal{L}_{i,k+1}|}. \quad (41)$$

The weights are then computed as a moving average of the form (Eq. (42)), where α is a constant:

$$\lambda_{i,k+1} = (1 - \alpha) \lambda_{i,k} + \alpha \tilde{\lambda}_{i,k+1}. \quad (42)$$

Algorithm 1 presents the ALW pseudocode implemented for the present work. At every iteration k , up to the last iteration N_k , the unweighted loss components are first calculated (line 3). The core of the adaptive loss weighting lies between lines 4 and 14, which is performed only every δ_1 iterations. This is important to avoid an excessive computational burden, as every ALW step requires $N_{\mathcal{L}} - 1$ additional backpropagation steps (one for each loss component gradient, except the first

Algorithm 1 Adaptive loss weights. Adapted from [7], [15].

```

1: Initialize  $\lambda_{k=1} = [1, 1, \dots, 1]^T$ 
2: for  $k = 1, \dots, N_k$  do
3:    $\tilde{\mathcal{L}}_{k+1} = [\tilde{\mathcal{L}}_{1,k+1}, \tilde{\mathcal{L}}_{2,k+1}, \dots, \tilde{\mathcal{L}}_{N_{\mathcal{L}},k+1}]^T$ 
4:   if  $k - 1 \bmod \delta_1 = 0$  then
5:     for  $i = 1, \dots, N_{\mathcal{L}}$  do
6:        $\tilde{\lambda}_{i,k+1} = \frac{|\nabla_{\theta} \tilde{\mathcal{L}}_{1,k+1}|}{|\nabla_{\theta} \tilde{\mathcal{L}}_{i,k+1}|}$ 
7:     end for
8:      $\tilde{\lambda}_{k+1} = [\tilde{\lambda}_{1,k+1}, \tilde{\lambda}_{2,k+1}, \dots, \tilde{\lambda}_{N_{\mathcal{L}},k+1}]^T$ 
9:     if  $k = 1$  then
10:       $\lambda_{k+1} = \tilde{\lambda}_{k+1}$ 
11:     else
12:       $\lambda_{k+1} = (1 - \alpha)\lambda_{k+1-\delta_1} + \alpha\tilde{\lambda}_{k+1}$ 
13:     end if
14:      $\mathcal{L}_{k+1} = \lambda_{k+1} \odot \tilde{\mathcal{L}}_{k+1}$ 
15:   else
16:      $\mathcal{L}_{k+1} = \lambda_k \odot \tilde{\mathcal{L}}_{k+1}$ 
17:   end if
18:    $\mathcal{L}_{k+1} = [\mathcal{L}_{1,k+1}, \mathcal{L}_{2,k+1}, \dots, \mathcal{L}_{N_{\mathcal{L}},k+1}]^T$ 
19:    $\mathcal{L}_{k+1} = \sum_{i=1}^{N_{\mathcal{L}}} \mathcal{L}_{i,k+1}$ 
20:   Backpropagate  $\mathcal{L}_{k+1}$ 
21: end for

```

one). Consequently, the update interval δ_1 represents a trade-off between computational cost and the adaptive weighting response to gradient changes as training progresses. Besides, computing the weights as a moving average of their previous values smooths out the high variance that instantaneous gradient values are expected to present, given the stochastic nature of the gradient descent updates [15]. In the end (line 20), the total loss $\mathcal{L}_{k+1}$ is then backpropagated, starting the optimization step using Adam.

B. Hardware and Software Setup

The computations for the present work are performed in single precision, using a single GPU for training and inferencing. To apply the PINNs algorithm, a Python code was written, based on the DeepXDE framework [16], and the backend for the machine learning operations is PyTorch [17]. Table I resumes the main hardware and software specifications of the equipment used to perform the tests in this report.

TABLE I: Overview of the hardware and software employed in the present work.

Architecture	x86-64 (virtualized)
CPU model	Intel Xeon Silver 4214R
Memory	256 GB RAM
GPU	Nvidia Tesla V100S PCIe 32 GB
Nvidia driver version	555.42.02
CUDA version	12.6
Operating System	Ubuntu 20.04.6 LTS
ML Backend	PyTorch 2.9.1+cu126
PINNs Framework	DeepXDE 1.15.0

C. Problem Setup and Metrics

The case studies performed in this report, in section IV, are divided into two groups: V-PINNs and DT-PINNs. In each group, there are performed experiments with constant loss weights, equal to 1 (fixed weights) and with ALW technique (variable weights). To each one of these, there are considered hyperbolic tangent (tanh) and sigmoid activation functions. For each activation function, there are employed three different initial learning rates: $\eta_0 = 10^{-3}$, $\eta_0 = 10^{-4}$ and $\eta_0 = 10^{-5}$. In total, there are 24 combinations of case studies. In all cases, the learning rate, η , follows the trend given by Eq. (43):

$$\eta = \eta_0 C^k, \quad C = 0.1^{1/\delta_2}, \quad (43)$$

where k is the previous iteration. The learning rate is reduced by one order of magnitude at every δ_2 iterations, where $\delta_2 = 10000$ iterations for V-PINNs and DT-PINNs ($t = 0$) and $\delta_2 = 300$ iterations for DT-PINNs ($t > 0$).

The MLP for $t = 0$, in DT-PINNs, runs only with $\eta_0 = 10^{-3}$, also following Eq. (43), and its loss weights are always constant and equal to 1. In this case, given the relative simplicity of the loss function, applying ALW made little to no difference in terms of accuracy.

For every case study, $\text{Re} = a = c = 1$, the spatial domain is $\Omega_s = [-1, 1] \times [-1, 1] \times [-1, 1]$ and the temporal domain is $\Omega_t = [0, 1]$ with time steps $dt = 0.01$, meaning that there are 101 time steps. The optimizer is Adam [11] and the network parameters are initialized with Glorot Normal [18].

For the ALW technique: the moving average constant is $\alpha = 0.1$ (Eq. (42)) and the update interval is $\delta_1 = 100$ iterations.

Each training set $\mathcal{T}_i$ consists, in every case study, of $N_{\mathcal{T}} = 1000$ randomly distributed points (for V-PINN, $N_{\mathcal{T}} = N_f + N_{BC} + N_{IC}$: $N_f = 700$, $N_{BC} = 150$ and $N_{IC} = 150$ points; for DT-PINN, $N_{\mathcal{T}} = N_f + N_{BC}$: $N_f = 800$, $N_{BC} = 200$ points). Besides, the test set consists, in every case study, on the same structured grid of $32^3 = 32768$ 3D-space points per time instant (total of $32768 \times 101 = 3,309,568$ space-time points).

For DT-PINNs, the MLP is trained with 60000 iterations ($t = 0$) plus 300 iterations per time step ($t > 0$), in a total of $60000 + 100 \times 300 = 90000$ iterations; when the training for the last iteration at each time step ends, the model stops training, being inferenced for that time step. For V-PINNs, the stopping criterion is defined based on the average wall-clock time, $\overline{t_{\text{avg}}}$, that an equivalent DT-PINN takes to train. As soon as a given V-PINN, at its training stage, reaches a wall-clock time of $\overline{t_{\text{avg}}}$, the current iteration completes its training, and the model is inferenced. There are four different scenarios, and $\overline{t_{\text{avg}}} = \{\overline{t_1}, \overline{t_2}, \overline{t_3}, \overline{t_4}\}$, depending on the scenario. The average wall-clock time (through all training sets and learning rates) for DT-PINNs + fixed weights + tanh is equal to $\overline{t_1}$. All V-PINN models with fixed weights and tanh must stop training when the iteration coincident with training wall-clock time $\overline{t_1}$ finishes. The same reasoning applies to all V-PINNs + fixed weights + sigmoid ($\overline{t_2}$), all V-PINNs + variable weights + tanh ($\overline{t_3}$), and all V-PINNs + variable weights + sigmoid ($\overline{t_4}$). This allows for a fair comparison between groups of V-PINNs and DT-PINNs, evaluating their model accuracy given the same computing time constraints.

All case studies were run with deterministic mode activated, ensuring that multiple runs with the same inputs, parameters and architecture lead to the same outputs. This eliminates the effect of non-determinism on the model accuracy and performance. There were produced two groups of $\mathcal{N}_T = 10$ different training sets each, one of them common to all V-PINN case studies, and the other, common to all DT-PINN case studies. A group of training sets is defined as $\mathbb{T} = \{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_{\mathcal{N}_T}\}$.

In subsections IV-A and IV-B, each case study's convergence was evaluated using the ten-run average total unweighted loss, $\bar{\mathcal{L}}$, ten-run average unweighted residual loss, $\bar{\mathcal{L}}_f$, ten-run average unweighted BC loss, $\bar{\mathcal{L}}_{BC}$, and ten-run average unweighted IC loss, $\bar{\mathcal{L}}_{IC}$ (this one only for V-PINNs). Letting $\bar{\mathcal{L}}_X$ be any of the quantities $\bar{\mathcal{L}}_f$, $\bar{\mathcal{L}}_{BC}$, $\bar{\mathcal{L}}_{IC}$, one has Eq. (44), for each iteration k :

$$\bar{\mathcal{L}}_X(k) = \frac{1}{\mathcal{N}_T} \sum_{r=1}^{\mathcal{N}_T} \tilde{\mathcal{L}}_X(r, k), \quad (44)$$

where $\tilde{\mathcal{L}}_X(r, k)$ is the unweighted X-component loss function at iteration k of run r for the training set $\mathcal{T}_i$, and $\bar{\mathcal{L}}$ is the sum of all $\bar{\mathcal{L}}_X$ components.

The minimum and maximum total losses across all training sets at iteration k are, respectively, Eq. (45, 46):

$$\tilde{\mathcal{L}}_{\min}(k) = \min_{i=1, \dots, \mathcal{N}_T} \tilde{\mathcal{L}}(r, k), \quad (45)$$

$$\tilde{\mathcal{L}}_{\max}(k) = \max_{i=1, \dots, \mathcal{N}_T} \tilde{\mathcal{L}}(r, k). \quad (46)$$

Let $y(j, r, t)$ and $\hat{y}(j, r, t)$ be any of the four physical quantities u, v, w, p , exact, and $\hat{u}, \hat{v}, \hat{w}, \hat{p}$, predicted at the j -th test point, run r , time t , within a test set containing N_S points. The L_2 relative error for y , $L_{2,y}$, and the residuals average, f_{avg} , at a run r and time instant t , are defined as, respectively (Eqs. (47), (48)):

$$L_{2,y}(r, t) = \sqrt{\frac{\sum_{j=1}^{N_S} (y(j, r, t) - \hat{y}(j, r, t))^2}{\sum_{j=1}^{N_S} (y(j, r, t))^2}}, \quad (47)$$

$$f_{\text{avg}}(r, t) = \frac{1}{4N_S} \sum_{i=1}^4 \sum_{j=1}^{N_S} |f_i(j, r, t)|, \quad (48)$$

and the respective ten-run averages, at time instant t are (Eqs. (49), (50)):

$$\overline{L_{2,y}}(t) = \frac{1}{\mathcal{N}_T} \sum_{r=1}^{\mathcal{N}_T} L_{2,y}(r, t), \quad (49)$$

$$\overline{f_{\text{avg}}}(t) = \frac{1}{\mathcal{N}_T} \sum_{r=1}^{\mathcal{N}_T} f_{\text{avg}}(r, t). \quad (50)$$

IV. RESULTS AND DISCUSSION

A. Accuracy and Convergence: V-PINNs

The first, second, third and fourth rows of plots from Fig. 2 show, respectively, the evolution of the ten-run-averaged total unweighted loss, $\bar{\mathcal{L}}$, ten-run-averaged unweighted residual loss, $\bar{\mathcal{L}}_f$, ten-run-averaged unweighted BC loss, $\bar{\mathcal{L}}_{BC}$, and ten-run-averaged unweighted IC loss, $\bar{\mathcal{L}}_{IC}$, for twelve different

hyperparameter combinations. Starting with the initial learning rates: $\eta_0 = 10^{-3}$ (left plot), $\eta_0 = 10^{-4}$ (center plot) and $\eta_0 = 10^{-5}$ (right plot). At each plot, one has fixed weights and tanh (black solid lines), fixed weights and sigmoid (red solid lines), variable weights and tanh (green solid lines), and variable weights and sigmoid (blue solid lines). Also, in the first row of plots, each curve is surrounded by a shaded area of the same color, which represents, for that specific hyperparameter combination, the range between the minimum and maximum values of $\bar{\mathcal{L}}$ across all training sets at each iteration, $\tilde{\mathcal{L}}_{\min}$ and $\tilde{\mathcal{L}}_{\max}$, respectively.

For $\eta = 10^{-3}$, between the first few iterations up to around iteration 15000, there are some noticeable oscillations during model training, which can be attributed to the fact that in this stage the parameter updates are relatively large, causing the optimizer to jump back and forth across the loss landscape, rather than following a smooth loss convergence trajectory, suggesting that the learning rate η is relatively high at early stage training. In fact, these instabilities are almost not present at $\eta = 10^{-4}$ and are inexistent at $\eta = 10^{-5}$. However, despite the local instabilities when $\eta = 10^{-3}$, globally, the loss keeps reducing and converging. The lowest values for $\bar{\mathcal{L}}$ at the last iteration are obtained for the PINN trained with $\eta_0 = 10^{-3}$ + variable weights + tanh, reaching a value of 6.34×10^{-5} , which is very closely followed by fixed weights + tanh and fixed weights + sigmoid, both also with $\eta_0 = 10^{-3}$, performing better than the more stable runs of the remaining η_0 values. For $\eta_0 = 10^{-4}$ and $\eta_0 = 10^{-5}$, the lowest $\bar{\mathcal{L}}$ were 2.02×10^{-4} with fixed weights + sigmoid, and 2.38×10^{-3} , with fixed weights + tanh, respectively.

The introduction of the ALW technique to all combinations of learning rate and activation function tested resulted, actually, in a significant increase of the average unweighted loss values at any iteration, with the exception of $\eta_0 = 10^{-3}$ with tanh, where ALW made slight improvements across most iterations, compared to fixed weights. This issue is especially noticeable with the V-PINNs trained with sigmoid (blue lines), where, besides the very high values for $\bar{\mathcal{L}}$, at any of the learning rates tested (at the last iteration, $\bar{\mathcal{L}} = \{5.17 \times 10^0, 1.47 \times 10^{-1}, 5.44 \times 10^0\}$ for $\eta_0 = \{10^{-3}, 10^{-4}, 10^{-5}\}$), one can also see that the loss evolution is highly dependent on the training set, seen by the large blue-shaded area, which is a signal of high instability and poor model reproducibility. For example, in the last iteration, for the combination of variable weights, sigmoid, and $\eta_0 = 10^{-3}$, the metrics ($\tilde{\mathcal{L}}_{\min}, \bar{\mathcal{L}}, \tilde{\mathcal{L}}_{\max}$) are equal to $(7.46 \times 10^{-4}, 5.17 \times 10^0, 7.76 \times 10^0)$. The proximity between $\bar{\mathcal{L}}$ and $\tilde{\mathcal{L}}_{\max}$ indicates that most runs ended up with total loss values at the last iteration close to the maximum, with the exception of two outliers (one is the minimum ($\tilde{\mathcal{L}}_{\min}$), the other is a $\bar{\mathcal{L}}$ value of the order of 10^{-2}).

To properly evaluate the training process, one cannot look only for the total loss function; its components also need to be analysed: residual, BC and IC losses. The distribution of $\bar{\mathcal{L}}$ between its components is more or less balanced in the V-PINNs correspondent to the green, red and black lines: within the same hyperparameter combination, the difference between

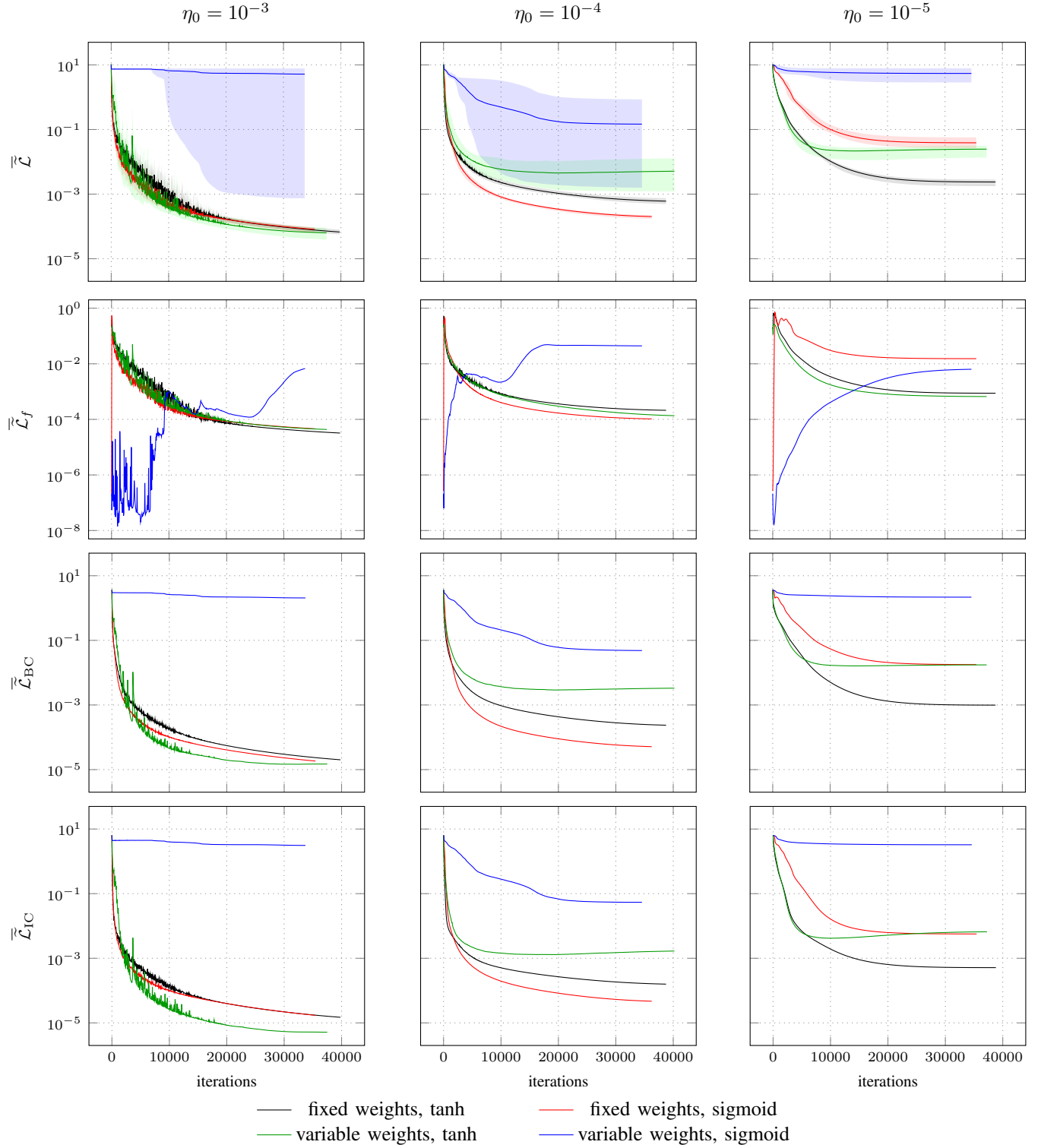


Fig. 2: Evolution of the ten-run average total loss function and the respective loss components for V-PINNs, in function of the number of training iterations, for fixed weights + tanh (black line), fixed weights + sigmoid (red line), variable weights + tanh (green line), and variable weights + sigmoid (blue line).

different loss components, at any given iteration, is within one order of magnitude. However, in the PINNs correspondent to the variable weights + sigmoid, while $\bar{\mathcal{L}}_{BC}$ and $\bar{\mathcal{L}}_{IC}$ are relatively close between each other at any iteration k , $\bar{\mathcal{L}}_f$ shows a different trend than the remaining loss components.

At the beginning of training, $\bar{\mathcal{L}}_f$ is several orders of magnitude smaller than $\bar{\mathcal{L}}_{BC}$ and $\bar{\mathcal{L}}_{IC}$. As training progresses, the residual loss increases by about four to five orders of magnitude, while the boundary and initial-condition losses barely decrease, when $\eta_0 = 10^{-3}$ and $\eta_0 = 10^{-5}$. This behaviour indicates that

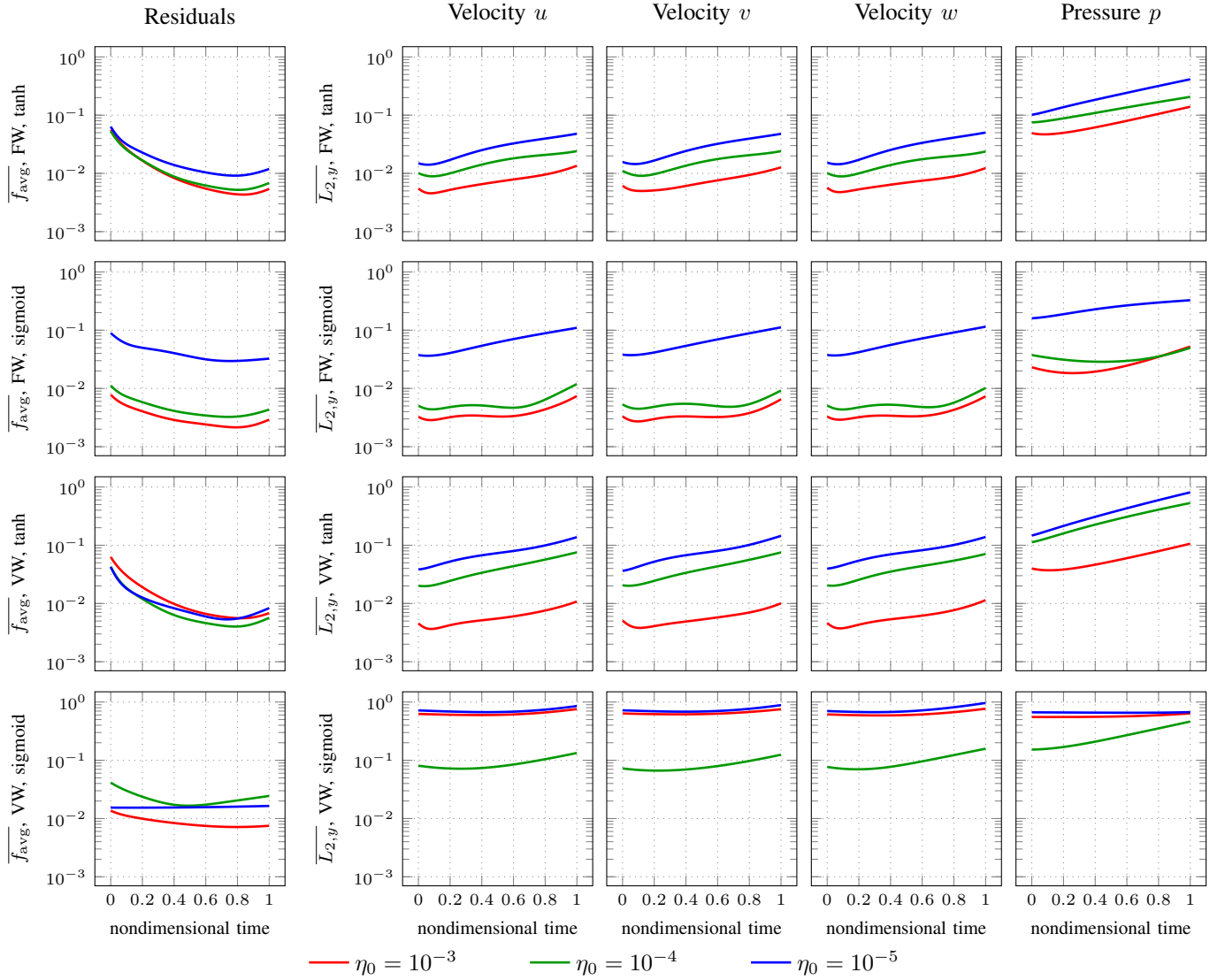


Fig. 3: For each η_0 considered (red line: $\eta_0 = 10^{-3}$, green line: $\eta_0 = 10^{-4}$, blue line: $\eta_0 = 10^{-5}$), for V-PINNs, these are the ten-run average residuals average, $\overline{f_{\text{avg}}}$ (column "Residuals") and ten-run average L_2 relative errors (remaining columns) evolution in function of the nondimensional time. From top to bottom: fixed weights (FW) + tanh, FW + sigmoid, variable weights (VW) + tanh, VW + sigmoid. From left to right: residuals, velocity u , velocity v , velocity w , and pressure p .

the network initially represents a solution that nearly satisfies the PDE residual but hardly satisfies the BC and IC at any stage of training when $\eta_0 = 10^{-3}$ and $\eta_0 = 10^{-5}$.

Figure 3 presents the evolution of the ten-run averaged residuals average ($\overline{f_{\text{avg}}}$), at the first column of plots, and the ten-run averaged L_2 errors for velocities u ($\overline{L_{2,u}}$), v ($\overline{L_{2,v}}$), and w ($\overline{L_{2,w}}$), and pressure p ($\overline{L_{2,p}}$), at the second, third, fourth and fifth columns of plots, respectively. The tests performed with fixed weights, tanh; fixed weights, sigmoid; variable weights, tanh; and variable weights, sigmoid, are presented at the first, second, third and fourth rows of plots. Finally, each plot contains three lines, representing the evolution of the respective metrics throughout the nondimensional time considering $\eta_0 = 10^{-3}$ (red), $\eta_0 = 10^{-4}$ (green), and $\eta_0 = 10^{-5}$ (blue). To facilitate the comparison between the curves of different plots, it was established a unique scale for

the horizontal and vertical axis, shared between all plots.

The residuals tend to decrease as t rises, up to around $t = 0.8$, after which it shows a slight increase, except for variable weights + sigmoid combinations. The L_2 errors increase as t progresses, which can be seen as a consequence of the temporal causality infringement explained in subsection II-D. In such cases, PINNs may tend to minimize the PDE residuals at later periods before properly learning the solutions at earlier times [19]. This can be verified when comparing the temporal evolution of $\overline{f_{\text{avg}}}$ with $\overline{L_{2,y}}$ at the first, second and third rows of plots in Fig. 3: $\overline{f_{\text{avg}}}$ decreases with time, up to around $t = 0.8$ to $t = 0.85$, after which it starts to increase, while $\overline{L_{2,y}}$ generally increases with time.

To facilitate the comparison of $\overline{L_{2,y}}$ and $\overline{f_{\text{avg}}}$ between the hyperparameters and PINN techniques tested, it may be appropriated to compute their respective averages over the

TABLE II: Average values for $\overline{f_{\text{avg}}}$, $\overline{L_{2,u}}$, $\overline{L_{2,v}}$, $\overline{L_{2,w}}$, and $\overline{L_{2,p}}$ for all V-PINNs case studies. The lowest values for each row are in bold and underlined. The values presented must be multiplied by 10^{-3} .

Metric / η_0	Fixed Weights						Variable Weights					
	tanh			sigmoid			tanh			sigmoid		
	10^{-3}	10^{-4}	10^{-5}	10^{-3}	10^{-4}	10^{-5}	10^{-3}	10^{-4}	10^{-5}	10^{-3}	10^{-4}	10^{-5}
$\overline{f_{\text{avg}}}$ [10^{-3}]	11.64	11.97	17.07	<u>3.22</u>	4.67	41.04	13.60	8.92	10.16	8.59	21.85	15.69
$\overline{L_{2,u}}$ [10^{-3}]	7.57	15.87	28.95	<u>3.83</u>	5.74	65.06	5.99	40.95	76.78	632.63	87.83	712.03
$\overline{L_{2,v}}$ [10^{-3}]	7.39	15.62	28.62	<u>3.54</u>	5.44	65.23	5.72	40.77	77.73	646.39	81.76	728.01
$\overline{L_{2,w}}$ [10^{-3}]	7.26	15.48	29.72	<u>3.82</u>	5.48	67.03	6.16	40.63	78.73	630.17	96.07	741.07
$\overline{L_{2,p}}$ [10^{-3}]	77.47	127.85	226.34	<u>26.68</u>	33.38	244.85	58.85	283.43	403.02	573.66	260.28	660.22

time domain: Tab. II presents the average values of each line presented at the plots of Fig. 3. It shows that the introduction of the ALW technique for any given hyperparameter combination generally decreased model accuracy when compared to fixed weights (equal to 1) at the same hyperparameter combination, except for the combination of tanh + $\eta_0 = 10^{-3}$, where $\overline{L_{2,y}}$ slightly decreased, on average, which is expected given the respective loss values at the last iteration. The V-PINN that presented the worst predictions has the combination variable weights + sigmoid + $\eta_0 = 10^{-5}$, with averages $\overline{L_{2,y}}$ of between 6.60×10^{-1} and 7.41×10^{-1} , reaching a maximum of $\overline{L_{2,p}} = 9.61 \times 10^{-1}$ at $t = 1$, all values being quite high and not acceptable for accurate predictions. These results are coincident with the fact that it was this exact combination that presented the highest $\tilde{\mathcal{L}}_{\text{BC}}$ and $\tilde{\mathcal{L}}_{\text{IC}}$, closely followed by the combination variable weights + sigmoid + $\eta_0 = 10^{-3}$. The best results for any given metric, for V-PINN, are in bold and underlined, and were all achieved by the combination fixed weights + sigmoid + $\eta_0 = 10^{-3}$, with averages $\overline{L_{2,y}}$ of between 3.54×10^{-3} and 2.67×10^{-2} , $\overline{f_{\text{avg}}} = 3.22 \times 10^{-3}$, minimum and maximum $\overline{f_{\text{avg}}}$ equal to 2.15×10^{-3} at $t = 0.79$ and 7.81×10^{-3} at $t = 0$, respectively, and minimum and maximum $\overline{L_{2,y}}$ of $\overline{L_{2,v}} = 2.72 \times 10^{-3}$ at $t = 0.1$ and $\overline{L_{2,p}} = 5.25 \times 10^{-2}$ at $t = 1$, respectively. The accuracies for fixed weights + sigmoid + $\eta_0 = 10^{-3}$ are better than the combination variable weights + tanh + $\eta_0 = 10^{-3}$ (but within the same order of magnitude), while the loss values shown by the correspondent lines and plots of Fig. 2 (first column of plots: red and green lines, respectively) show the opposite, but by a relatively low margin, which is acceptable, given that the losses are computed on the training set, which is quite different from the test set, on which the metrics in Fig. 3 and Tab. II are produced.

B. Accuracy and Convergence: DT-PINNs

Figure 4 shows the evolution of the ten-run average total loss function, $\tilde{\mathcal{L}}$, for the MLP correspondent to $t = 0$, in the two case studies considered: with tanh and with sigmoid as activation functions. It is important to remember that for $t = 0$ the models trained are not PINNs, but rather, are conventional MLPs trained to respect the initial condition at the training points. The MLP for $t = 0$ should be very well trained to maximize accuracy not only for this initial step but also for the subsequent steps at $t > 0$. Besides, for $t = 0$, the loss weights

are always constant and equal to 1. After repeated experiments, it was verified that, for both tanh and sigmoid, the total loss reaches a plateau after around 50000 to 60000 iterations, converging to values below 10^{-5} , so it was decided to stop training at the 60000th iteration. Also, in Fig. 4, between the first few iterations up to around 18000 iterations, there are present the same kind of instabilities explained in subsection IV-A and seen in the left plots of Fig. 2. Despite these, the loss keeps reducing and converging, as expected, reaching, as an average of the ten runs, a final value of 3.63×10^{-6} for tanh and 5.92×10^{-6} for sigmoid.

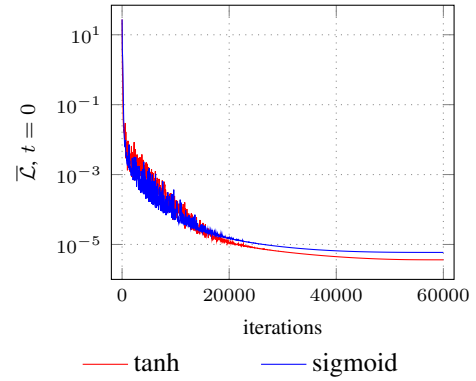


Fig. 4: Ten-run average total loss function, with tanh (red line) and sigmoid (blue line), in function of the iterations, for DT-PINN, $t = 0$.

Figure 5 presents, for $t > 0$, the evolution of the ten-run average residual unweighted loss, $\tilde{\mathcal{L}}_f$, at the top row of plots, and the evolution of the ten-run average BC unweighted loss, $\tilde{\mathcal{L}}_{\text{BC}}$, at the bottom row of plots. At each of the three columns of plots (left: $\eta_0 = 10^{-3}$, middle: $\eta_0 = 10^{-4}$, right: $\eta_0 = 10^{-5}$), there are the lines correspondent to the above-mentioned metrics with fixed weights + tanh (black), fixed weights + sigmoid (red), variable weights + tanh (green), and variable weights + sigmoid (blue).

The highly oscillatory behaviour that can be observed at almost all loss curves from Fig. 5 show, during training, is due to the specific training structure of DT-PINNs. As explained in subsection II-D, for $t > 0$, there are trained N PINNs (where N is the number of time steps), sequentially. Each PINN corresponds to a time step $t^{(n)}$ which is trained in 301 iterations.

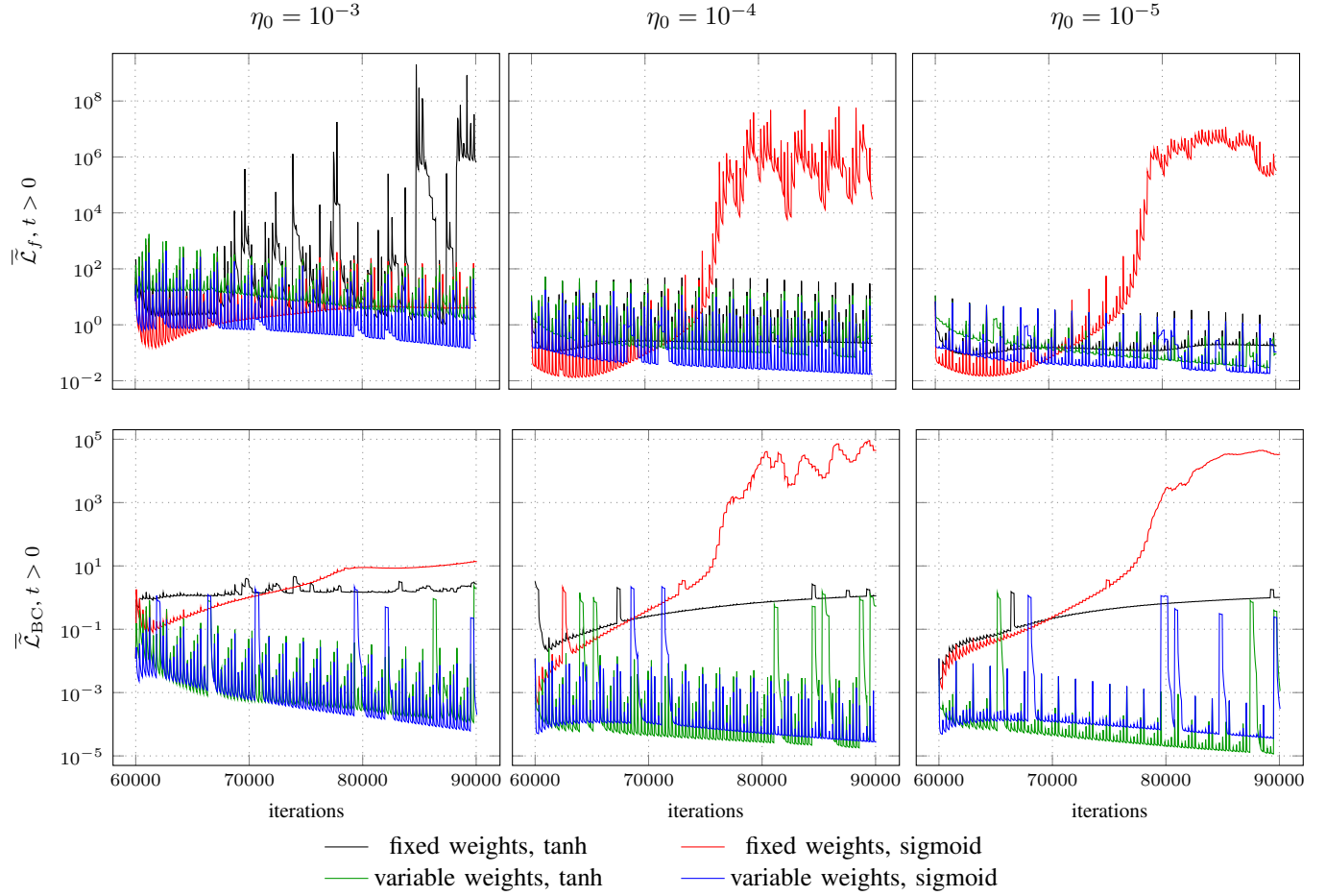


Fig. 5: Evolution of the ten-run average unweighted residual loss (top row of plots) and ten-run average unweighted BC loss (bottom row), in function of the number of training iterations, for fixed weights + tanh (black line), fixed weights + sigmoid (red line), variable weights + tanh (green line), and variable weights + sigmoid (blue line), for DT-PINN ($t > 0$).

Right after these iterations are completed, the trained model is inferred at the test set and the corresponding parameters $\theta^{(n)}$ from the last iteration, $k = k_f = 300$, are initialized at the first iteration, $k = k_0 = 0$, of the PINN from the next time step, thus, $\theta^{(n)}(k_f) = \theta^{(n+1)}(k_0)$.

Figure 6 is a detailed view of the portion of the bottom-right plot of Fig. 5 showing $\bar{\mathcal{L}}_{BC}$ with $\eta_0 = 10^{-5}$, from iteration $k = 60000$ to $k = 62250$, which perfectly exemplifies the oscillatory behaviour mentioned above of the loss components. It shows that at the beginning of every PINN $n + 1$ the loss values are relatively high (local maximum), which is expected, given that it corresponds to a model trained on parameters from the PINN n . However, as the time step $\Delta t = 0.01$ is small relatively to the slow dynamics of the Beltrami flow, the solution changes only slightly between consecutive time steps. Therefore, the parameters transferred from PINN n already provide a reasonably good initial approximation for PINN $n + 1$, requiring only a few hundred optimization iterations to reach a local minimum. The top side of the plot from Fig. 6 shows the inference points, corresponding to the time instants indicated, which are often coincident (or close) to the local minima of the loss for the respective PINN (with the exception of the hyperparameters that lead to chaotic training behaviour,

explained in the next paragraph). For example, PINN 1, for $t = 0.01$, was trained from global iteration 60000 to 60300, and then inferred (at the test set) right after iteration 60300 was completed. Then, PINN 2, for $t = 0.02$, was trained from the global iteration 60301 to 60601, being inferred right after training from iteration 60601 ends, and so on for the subsequent PINNs up to the last time step.

Returning back to Fig. 5, the fixed weights + sigmoid curves exhibit catastrophic behaviour, meaning that $\bar{\mathcal{L}}_{BC}$ and $\bar{\mathcal{L}}_f$ diverge to extremely high values, where the local minima are $\bar{\mathcal{L}}_f > 10^4$ and $\bar{\mathcal{L}}_{BC} > 10^3$ after iteration 80000, for $\eta_0 = 10^{-4}$, and $\bar{\mathcal{L}}_f > 10^5$ and $\bar{\mathcal{L}}_{BC} > 10^3$ for $\eta_0 = 10^{-5}$, also after around iteration 80000, for example, despite that in the first few thousand iterations, these losses' local minima hovered $\mathcal{O}(10^{-2})$, which is relatively low. The training process was, in general, quite unsuccessful with DT-PINNs using fixed weights, since after about iteration 75000, both loss components are almost always above 1, which is already too high (except for $\bar{\mathcal{L}}_f$ with $\eta_0 = 10^{-3}$). The introduction of the ALW technique to the DT-PINNs significantly improves the quality of training and, at almost every time step, apart from a few outliers, $\bar{\mathcal{L}}_{BC}$ reaches local minima below 10^{-3} (except

for up to iteration 65000 with $\eta_0 = 10^{-5}$), approaching an absolute minimum value of $\bar{\mathcal{L}}_{BC} = 1.15 \times 10^{-5}$ at iteration 89498, right before inferencing at $t = 0.98$. The loss $\bar{\mathcal{L}}_f$, with variable weights, has its local minima ranging from $\mathcal{O}(10^{-2})$ to $\mathcal{O}(10^0)$ for sigmoid and from $\mathcal{O}(10^{-2})$ to $\mathcal{O}(10^1)$ (losses of 10^0 and 10^{-1} are already too high, indicating that the models tend to fit the boundary conditions better than the physical equations).

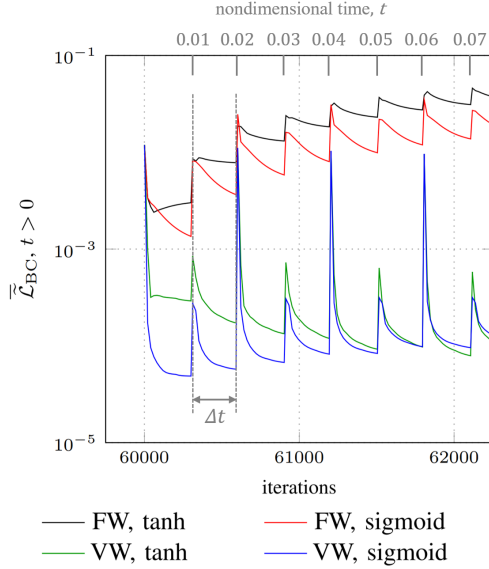


Fig. 6: Detailed view of the plot from Fig. 5 that shows the evolution of $\bar{\mathcal{L}}_{BC}$ with $\eta_0 = 10^{-5}$, from iteration 60000 to 62250.

Figure 7 and Tab. III are identical to Fig. 3 and Tab. II, respectively, but applied to DT-PINNs. At Fig. 7 it was also established a unique scale for the horizontal and vertical axis, shared between all plots. Every DT-PINN tested registered their absolute minimum value for $\bar{L}_{2,y}$ at $t = 0$, which was to be expected, given that $t = 0$ was very well trained and the loss function was relatively easy to minimize. In that scenario, $\bar{L}_{2,y} = [1.50 \times 10^{-3}, 1.90 \times 10^{-3}]$, for models tested with tanh, and $\bar{L}_{2,y} = [6.08 \times 10^{-4}, 8.76 \times 10^{-4}]$, for models tested with sigmoid, which are very good results. It is worth mentioning that at $t = 0$ there is no residual calculation because, as mentioned before, there are no governing equations in the loss function, since at this instant, the models are conventional MLPs trained to fit the initial condition.

Figure 7 also reveals that $\bar{f}_{avg}$ and $\bar{L}_{2,y}$ follow a similar trend to the local minima of the respective ten-run average loss functions seen at Fig. 5, despite the training and test sets being substantially different between each other. For example, one can highlight that, similarly to the respective losses, the DT-PINNs with fixed weights + sigmoid ($\eta_0 = 10^{-4}$ and $\eta_0 = 10^{-5}$) registered the highest difference between the highest and lowest values of $\bar{f}_{avg}$ and $\bar{L}_{2,y}$, reaching up to $\bar{f}_{avg} = 3.55 \times 10^2$ at $t = 0.84$ for $\eta_0 = 10^{-5}$, and up to $\bar{L}_{2,w} = 2.01 \times 10^2$ at $t = 0.97$ for $\eta_0 = 10^{-4}$, which are extremely high values, showing that the predicted velocity fields tend to diverge from the reference as time moves forward,

leading to completely non-physical results, as shown by the high residuals. In fact, the $\bar{L}_{2,y}$ values for most hyperparameter combination tested tend to climb as t increases, at rates which are highly dependent on the hyperparameters, especially for fixed weights, but also for variable weights, albeit at a lower rate (except for variable weights + sigmoid + $\eta_0 = 10^{-3}$, where $\bar{L}_{2,u}$, $\bar{L}_{2,v}$ and $\bar{L}_{2,w}$ are almost constant from low values of t up to $t = 1$), which may be explained by error propagation and accumulation between the sequential PINNs.

The curves for $\bar{L}_{2,y}$ and the respective average values from Tab. III confirm the fact that the introduction of ALW to DT-PINNs makes remarkable improvements to the accuracy of the models tested, especially when $\eta_0 = 10^{-4}$ and $\eta_0 = 10^{-5}$; for instance, from the data available at the table, one can easily find that for sigmoid, the introduction of this technique results in reductions of a factor between 517 and 11224 for $\eta_0 = 10^{-4}$ and between 717 and 10877 for $\eta_0 = 10^{-5}$ in the average of $\bar{L}_{2,y}$. Focusing on the third and fourth rows of plots of Fig. 7, for $\eta_0 = 10^{-3}$, all of the respective curves show a sharp increase on $\bar{L}_{2,y}$ during the first few instants (up to around $t = 0.03$ to $t = 0.08$, depending on the hyperparameter combination), of around one order of magnitude, significantly higher than what can be observed with the curves corresponding to the remaining values of η_0 , where that sharp increase is less pronounced. That accuracy degradation at the first steps may be explained by the abrupt transition from two completely different kinds of models between $t = 0$ (very well trained, conventional MLP and relatively easy to optimize) and $t > 0$ (sequential PINNs with much less iterations and relatively difficult to optimize, given the intricate loss components). After these initial steps, $\bar{L}_{2,y}$ gradually increases as t progresses, except for variable weights + sigmoid + $\eta_0 = 10^{-3}$, whose $\bar{L}_{2,y}$ keep approximately constant, apart from local peaks at six time instants.

C. Comparisons between V-PINN and DT-PINN

According to Tab. II, the hyperparameter combination that provided the lowest values for the averages of all five metrics tested, for the V-PINN configuration, was fixed weights + sigmoid + $\eta_0 = 10^{-3}$. Regarding DT-PINNs configuration, according to Tab. III, the combination variable weights + sigmoid + $\eta_0 = 10^{-4}$ presented the lowest average $\bar{f}_{avg}$, variable weights + tanh + $\eta_0 = 10^{-5}$ for the lowest averages of $\bar{L}_{2,u}$, $\bar{L}_{2,v}$ and $\bar{L}_{2,p}$, and variable weights + sigmoid + $\eta_0 = 10^{-5}$ for the lowest average of $\bar{L}_{2,w}$. Figure 8 compares the evolution of the above-mentioned best performing models for each metric, for the V-PINN (red line) and DT-PINN (blue line) configurations.

The V-PINN best performing model returned significantly lower residuals than DT-PINN (on average, the residuals from the best DT-PINN were about 17 times higher than the residuals from the best V-PINN model), which is a trend that also can be seen when analysing the first columns of plots from Fig. 3 and Fig. 7 or Tab. II and Tab. III. In fact, the V-PINN and DT-PINN ($t > 0$) models solve mathematically distinct problems: while V-PINNs solve directly the governing equations' derivatives in space and time through AD, DT-PINNs solve a second-order discrete time version of these equations, only applying

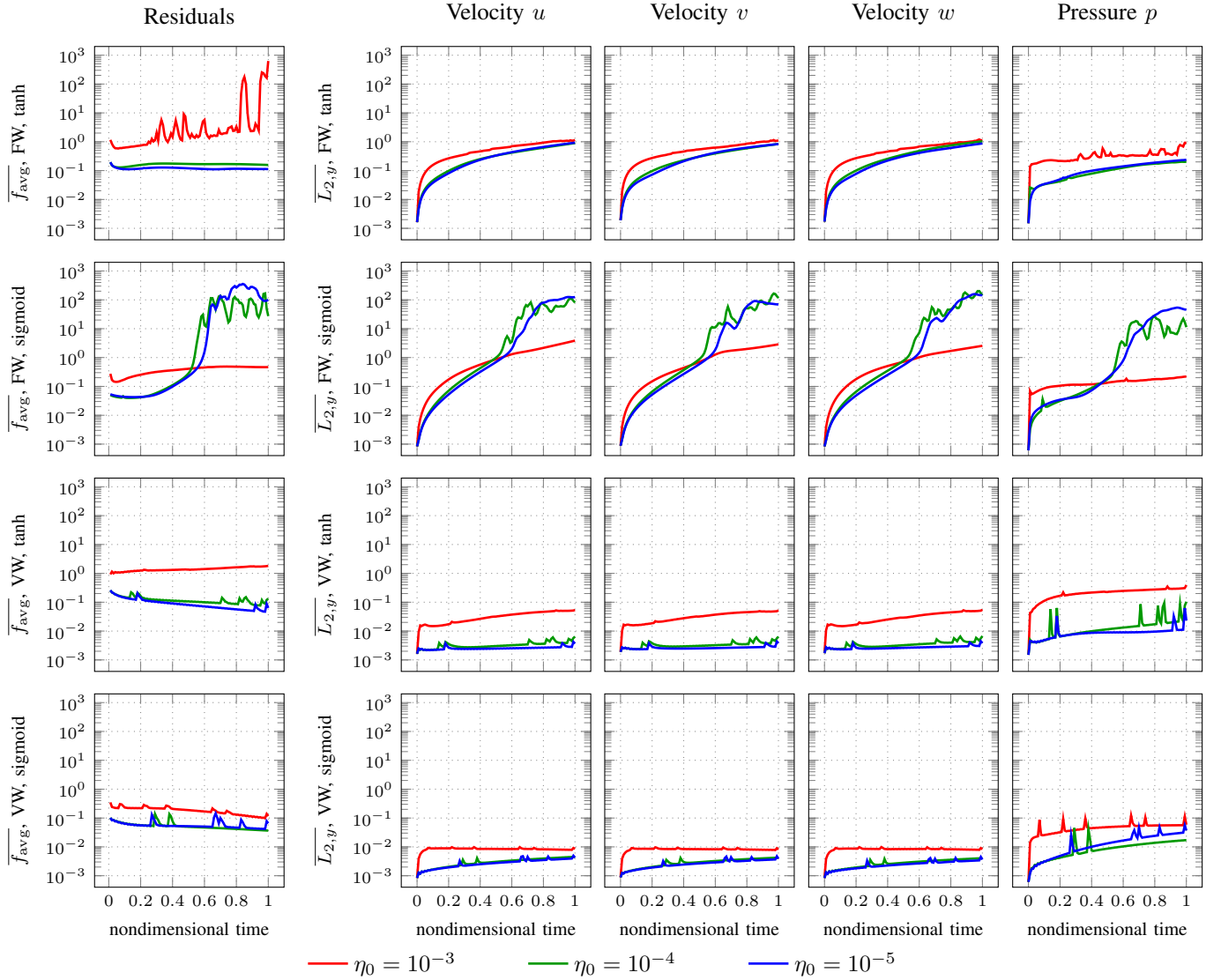


Fig. 7: For each η_0 considered (red line: $\eta_0 = 10^{-3}$, green line: $\eta_0 = 10^{-4}$, blue line: $\eta_0 = 10^{-5}$), for DT-PINNs, these are the ten-run average residuals average, $\bar{f}_{\text{avg}}$ (column "Residuals") and ten-run average L2 relative errors (remaining columns) evolution in function of the nondimensional time. From top to bottom: fixed weights (FW) + tanh, FW + sigmoid, variable weights (VW) + tanh, VW + sigmoid. From left to right: residuals, velocity u , velocity v , velocity w , and pressure p .

AD to the derivatives in space. Thus, their respective residuals (and residual losses) are also computed differently. A closer look to the best performing V-PINNs training processes reveals that the four components of the residual losses are around the same order of magnitude between each other (varying between $\mathcal{O}(10^{-5})$ and $\mathcal{O}(10^{-4})$ at the final iteration, depending on the hyperparameter combination), and that the best performing DT-PINNs have the three momentum-related residual losses (f_2 , f_3 and f_4) around the same order of magnitude ($\mathcal{O}(10^{-2})$ to $\mathcal{O}(10^{-1})$ for variable weights + $\eta_0 = 10^{-4}$ or $\eta_0 = 10^{-5}$, at the last iteration of each sequential PINN) between each other, while the continuity residual loss (f_1) lies between $\mathcal{O}(10^{-6})$ and $\mathcal{O}(10^{-4})$, also at the last iteration of each sequential PINN. The time discretization is only applied to the momentum equations, since these have a time derivative, unlike continuity, which is directly solved. As such, a possible

explanation for the discrepancy on the residuals between the best V-PINN and DT-PINN models may be somewhat related to the discretization performed. From the time derivative approximation from Eq. 34, and given that the predictions at time step n and $n + 1$ are equal to the exact results plus an approximation error ($e^{(n)}$, $e^{(n+1)}$) one has:

$$\frac{\hat{u}_i^{(n+1)} - \hat{u}_i^{(n)}}{\Delta t} = \frac{(u_i^{(n+1)} + e^{(n+1)}) - (u_i^{(n)} + e^{(n)})}{\Delta t}, \quad (51)$$

which is equivalent to:

$$\frac{\hat{u}_i^{(n+1)} - \hat{u}_i^{(n)}}{\Delta t} = \frac{u_i^{(n+1)} - u_i^{(n)}}{\Delta t} + \frac{\Delta e}{\Delta t}, \quad (52)$$

where $\Delta e = e^{(n+1)} - e^{(n)}$. Since $\Delta t = 0.01$, the approximation error difference between two consecutive iterations, Δe , is amplified by a factor of 100. The term $\Delta e / \Delta t$ is

TABLE III: Average values for $\overline{f_{\text{avg}}}$, $\overline{L_{2,u}}$, $\overline{L_{2,v}}$, $\overline{L_{2,w}}$, and $\overline{L_{2,p}}$ for all DT-PINNs case studies. The lowest values for each row are in bold and underlined. The values presented must be multiplied by 10^{-3} .

Metric / η_0	Fixed Weights						Variable Weights					
	tanh			sigmoid			tanh			sigmoid		
	10^{-3}	10^{-4}	10^{-5}	10^{-3}	10^{-4}	10^{-5}	10^{-3}	10^{-4}	10^{-5}	10^{-3}	10^{-4}	10^{-5}
$\overline{f_{\text{avg}}}$ [10^{-3}]	22059.14	161.42	117.86	370.73	29611.61	67977.34	1386.69	120.57	91.96	184.40	<u>54.09</u>	58.62
$\overline{L_{2,u}}$ [10^{-3}]	586.90	364.62	372.57	1192.73	22642.70	27182.93	32.15	3.35	<u>2.57</u>	8.38	3.02	2.80
$\overline{L_{2,v}}$ [10^{-3}]	556.01	344.07	346.67	1038.83	23511.74	19328.63	32.10	3.41	<u>2.65</u>	8.19	2.94	2.69
$\overline{L_{2,w}}$ [10^{-3}]	569.13	416.89	349.18	923.89	31387.62	27207.92	30.78	3.56	2.71	8.28	2.80	<u>2.50</u>
$\overline{L_{2,p}}$ [10^{-3}]	334.97	107.20	120.51	137.07	5124.05	11341.17	217.61	17.70	<u>9.48</u>	47.79	9.91	15.82

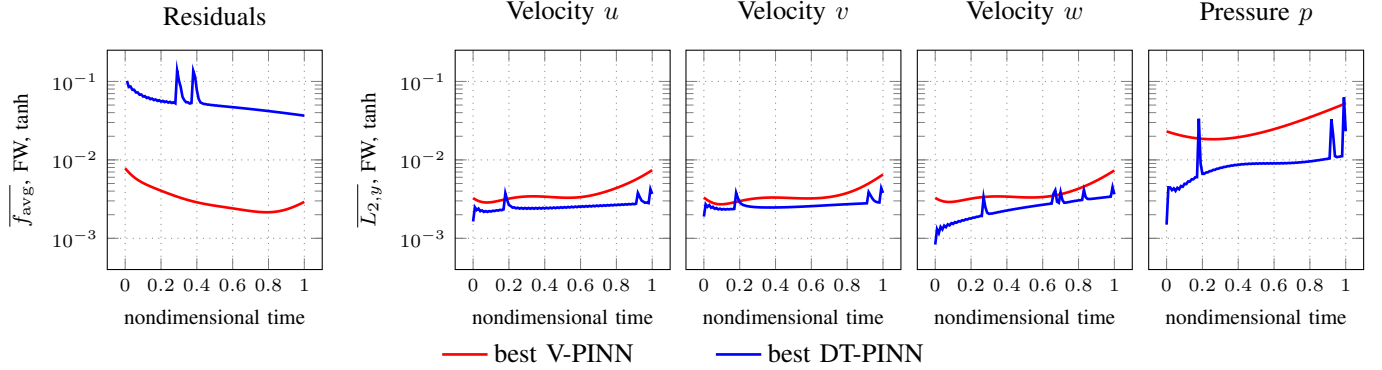


Fig. 8: Comparison of $\overline{f_{\text{avg}}}$ and $\overline{L_{2,y}}$ in function of the nondimensional time between the most accurate V-PINN model (red line) and DT-PINN models (blue line).

not present in the continuity equation and in any of the V-PINNs residuals. While $\Delta e / \Delta t$ may help explain the residual advantage of V-PINNs over DT-PINNs, more in-depth tests are required to confirm it; for example, by running the models on various values of Δt and studying their influence on $(\hat{u}_i^{(n+1)} - \hat{u}_i^{(n)}) / \Delta t$, measuring the actual values of $\Delta e / \Delta t$ and compare them with the remaining terms of the momentum equations.

Regarding $\overline{L_{2,y}}$, the best DT-PINNs, overall, return better predictions on the velocity and pressure fields when compared to the best V-PINN model, apart from a few peaks on certain time instants that also can be seen on the respective loss plots (see Fig. 5), indicating some degree of training instability, despite the significant improvements in model accuracy brought by the introduction of ALW to DT-PINNs. The average values for $\overline{L_{2,y}}$, $y = \{u, v, w\}$, are acceptable, varying between 3.54×10^{-3} and 3.83×10^{-3} for the best V-PINN and between 2.50×10^{-3} and 2.65×10^{-3} for the best DT-PINNs, while the pressure field is more difficult to predict, with averages $\overline{L_{2,p}} = 2.67 \times 10^{-2}$ for V-PINN and $\overline{L_{2,p}} = 9.48 \times 10^{-3}$ for DT-PINN.

D. Performance Analysis

Tables IV and V present, respectively, the average wall-clock times per iteration and peak Video Random Access Memory (VRAM) usage for the training process, for the V-PINN and DT-PINN models. For the latter there were separated the measurements for $t = 0$ and $t > 0$, since these are based on substantially different models. Besides, Tab. IV

shows the average number of iterations run for each group of case studies, right before inference. Training with sigmoid activation function requires more computational resources than training with tanh, both with V-PINN and DT-PINN ($t > 0$): with sigmoid, each iteration takes, on average, 58% to 61% longer than with tanh and requires 43% to 65% more VRAM.

The DT-PINN models are also computationally heavier than the V-PINN models, with a peak VRAM usage of 19% to 35% higher, depending on the weights and activation function. While each DT-PINN network is slightly smaller than a V-PINN network (input layer with three nodes instead of four), this architectural difference is outweighed by the additional tensors that must be retained between consecutive time steps. Specifically, before training the PINN corresponding to time step $n + 1$ (but after PINN n finishes its training), the spatial derivatives of u , v , w , and p at time step n are computed and stored for use in the residual loss evaluation on all iterations of the subsequent PINN. In addition, PyTorch Autograd [20] stores intermediate tensors required for these derivative computations. These additional memory requirements explain the observed increase in peak VRAM usage. This has a minor effect (within the margin of error) on the average time per iteration, with an increase of 1% to 4% with DT-PINNs ($t > 0$) compared to V-PINNs, as expected, since the extra gradient calculations are performed outside of the wall-clock time measurements, which only include the actual training stage.

When training PINNs with variable weights, the workload now includes the computation of the gradients of each loss

term with relation to the parameters at every $\delta_1 = 100$ iterations, in DT-PINNs ($t > 0$), which does not happen with V-PINNs; hence the slight increase in memory usage (between 2% to 6%) and in the average time per iteration (between 8% and 10%) verified in DT-PINNs ($t > 0$) in comparison to V-PINNs. Here, the extra calculations involved with the above-mentioned gradients are part of the training process and thus, incorporated in the wall-clock time measurements.

Regarding DT-PINNs, training the MLP model at $t = 0$ was a relatively light computation, at only between 15% and 22% of the memory requirements of the PINNs for $t > 0$, and taking only 9% to 15% of the average time per iteration needed for $t > 0$, depending on the weights (variable or fixed) and activation function. This is expected since the MLP for $t = 0$ has a much simpler loss function than the PINNs for $t > 0$, with fewer derivatives to compute, meaning faster, less complex and fewer backpropagation steps.

Overall, the memory requirements for training the models presented in this report are relatively modest, only requiring up to 2824 MB of VRAM, which most modern consumer-grade GPUs exceed.

TABLE IV: For V-PINN: average number of iterations, average wall-clock time per iteration and peak VRAM usage for the training process.

V-PINN	Fixed Weights		Variable Weights	
	tanh	sigmoid	tanh	sigmoid
Average number of iterations	39989	36754	39735	35873
Average time per iteration [ms]	88.57	140.08	95.22	153.69
Peak VRAM usage [MB]	1376	2256	1440	2380

TABLE V: For DT-PINN: average wall-clock time per iteration and peak VRAM usage for the training process, for $t = 0$ and $t > 0$.

DT-PINN	Fixed Weights		Variable Weights	
	tanh	sigmoid	tanh	sigmoid
Average time per iteration, $t = 0$ [ms]	13.26	13.31	13.57	13.72
Average time per iteration, $t > 0$ [ms]	91.24	144.50	98.64	155.82
Peak VRAM usage, $t = 0$ [MB]	426	434	426	434
Peak VRAM usage, $t > 0$ [MB]	1900	2720	1944	2824

V. CONCLUSION AND FUTURE WORK

This work focused on testing accuracy, convergence and performance of V-PINNs and DT-PINNs with and without ALW, on various hyperparameter combinations. These were successfully implemented, measured and compared, and thus, the main goal of this report was achieved.

However, these new techniques did not always show clear improvements regarding accuracy, despite introducing additional computational effort, especially regarding memory usage, when comparing with V-PINNs. For example, in V-PINNs, the case studies with variable weights registered, in almost all cases, worse average L_2 errors and residuals than the equivalent fixed weights configurations. The introduction of DT-PINNs with fixed weights resulted in chaotic accuracy results, and in this approach, implementing ALW was essential, since it brought significant stability and accuracy

improvements. Thus, in DT-PINNs, the most accurate results were found with variable weights. When comparing the most accurate V-PINNs and DT-PINNs case studies, one can conclude that the latter showed, in general, better accuracy than the former (apart from some peaks of higher errors at individual time steps), but a degradation on residuals average, which does not mean that the predictions from the best DT-PINN are less physical than those of the best V-PINN, since the source of these higher residuals may lie on the discretization method itself.

It is hard to directly compare the results here obtained to similar tests performed in the literature, because there are infinite hyperparameter combinations, training sets, MLP structures possible and this work shows that the model accuracy and convergence highly depends on these characteristics, besides the fact that there are many physics problems, and different methodologies for evaluation, which also influence the results. For example, [7] shows that the implementation of variable weights to V-PINNs generally increases model accuracy (i.e., lower L_2 relative errors) when predicting solutions for the Kovasznay flow (which is steady-state, 2D in space) and the flow past a circular cylinder (unsteady, 2D in space). Also, [13] demonstrates that DT-PINNs together with layer freezing, Fourier Features Embedding and modified MLPs (but with no variable weights) show significant reductions of between 139 and 2291 times, in average L_2 errors, in three transient, 1D-space PDEs, and a reduction of 38 times, in one 2D-space transient Navier-Stokes problem, compared to V-PINNs. However, it did not show that DT-PINNs together with any of the above-mentioned techniques are more efficient (i.e., higher number of iterations per unit of time) than V-PINNs. The governing equations of the problems in [7] and [13] are much easier to solve than the 3D-space transient Navier-Stokes in the present report, which may have influenced the results, besides the additional techniques employed in [13].

As previously stated, this work demonstrates that predicting reasonably accurate solutions for a 3D transient Navier-Stokes problem using MLP-based architectures strongly depends on the correct choice of hyperparameters. This time-consuming task increases overall inefficiency. Such limitations are expected to become even more pronounced when extending the methodology presented in this report to solve the same governing equations but applied to real-world scenarios, where analytical solutions do not exist, and more complex geometries are involved. Besides, there is a significant margin to improve accuracy and computational efficiency. The Navier-Stokes equations are a highly relevant benchmark for researching ML-based methods since these are expensive to solve and are the basis of many physical processes with diverse scientific and industry applications, such as weather forecasting, climate studies, aerospace, automotive, pipeline systems, and cardiovascular flows. Thus, future work should focus on researching alternative architectures to MLPs, more appropriate for space-time systems, aiming for increased efficiency and accuracy for solving 3D transient Navier-Stokes equations, targeting realistic, industry-based fluid mechanics problems. This would include, wherever possible, maximizing parallelization through distributed and parallel multi-GPU computing, comparing

the novel implementations against simulations obtained from conventional numerical methods, on accuracy, computational effort, and efficiency.

REFERENCES

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.10561>
- [2] —, “Physics Informed Deep Learning (Part ii): Data-driven Discovery of Nonlinear Partial Differential Equations,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.10566>
- [3] S. A. Faroughi, R. Soltanmohammad, P. Datta, S. K. Mahjour, and S. Faroughi, “Physics-informed neural networks with periodic activation functions for solute transport in heterogeneous porous media,” 2023. [Online]. Available: <https://arxiv.org/abs/2212.08965>
- [4] O. H. S. Posey, J. Luitjens and S. Oberlin, “GPU-Based HPC and AI Developments for CFD,” Maui, Hawaii, USA: NVIDIA Corporation, July 2022. [Online]. Available: https://www.iccfd.org/iccfd11/assets/pdf/papers/ICCFD11_Paper-3803.pdf
- [5] S. Wang, S. Sankaran, and P. Perdikaris, “Respecting causality is all you need for training physics-informed neural networks,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.07404>
- [6] D. Malveiro. (2025) GPU Parallelisation of a Two-Dimensional Navier-Stokes Solver with OpenACC. [Online]. Available: <https://fenix.tecnico.ulisboa.pt/cursos/memec21/dissertacao/1409728525633531>
- [7] X. Jin, S. Cai, H. Li, and G. E. Karniadakis, “NSFnets (Navier-Stokes Flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations,” *Journal of Computational Physics*, vol. 426, p. 109951, 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.jcp.2020.109951>
- [8] C. R. Ethier and D. A. Steinman, “Exact fully 3D Navier–Stokes solutions for benchmarking,” *International Journal for Numerical Methods in Fluids*, vol. 19, no. 5, pp. 369–375, 1994. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/flid.1650190502>
- [9] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, “DeepXDE: A Deep Learning Library for Solving Differential Equations,” *SIAM Review*, vol. 63, no. 1, p. 208–228, Jan. 2021. [Online]. Available: <http://dx.doi.org/10.1137/19M1274067>
- [10] K. Cai and J. Wang, “Physics-informed neural networks for solving incompressible Navier–Stokes equations in wind engineering,” *Physics of Fluids*, vol. 36, no. 12, p. 121303, 2024. [Online]. Available: <https://doi.org/10.1063/5.0244094>
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [12] Neural Concept. (2024) Physics-Informed Neural Networks in Engineering. Neural Concept. Accessed: 2026-06-23. [Online]. Available: <https://www.neuralconcept.com/post/physics-informed-neural-networks-in-engineering>
- [13] S. Chen, B. Shan, and Y. Li, “Efficient Discrete Physics-informed Neural Networks for Addressing Evolutionary Partial Differential Equations,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.14608>
- [14] J. H. Ferziger, M. Perić, and R. L. Street, *Computational Methods for Fluid Dynamics*, 4th ed. Cham, Switzerland: Springer, 2020.
- [15] S. Wang, Y. Teng, and P. Perdikaris, “Understanding and mitigating gradient pathologies in physics-informed neural networks,” 2020. [Online]. Available: <https://arxiv.org/abs/2001.04536>
- [16] L. Lu. (2019) DeepXDE. [Online]. Available: <https://deepxde.readthedocs.io/en/latest/index.html>
- [17] (2026) PyTorch. [Online]. Available: <https://pytorch.org/>
- [18] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 9, 2010, pp. 249–256.
- [19] M. Mullins, H. Kamil, A. Fahsi, and A. Soulaïmani, “Physics-informed neural networks for solving moving interface flow problems using the level set approach,” 02 2025.
- [20] PyTorch Contributors, *Automatic Differentiation Package – torch.autograd*, PyTorch Foundation, 2025, pyTorch 2.13 documentation. Accessed: 2026-07-22. [Online]. Available: <https://docs.pytorch.org/docs/2.13/autograd.html>